

Linux Sentinel — Screenshot Explanation & Presentation Playbook

A Student's Guide to Confidently Explaining Live Terminal Runs, Self-Healing, and Test Metrics to Your Technical Examiner

1. Executive Summary: What You Are Demonstrating

When presenting **Linux Sentinel** to your teacher or evaluator, your goal is to prove that this is a **real, deterministic C++17 user-space systems orchestrator**, not just a script or mockup. Traditional supervisors (such as `systemd`) only react *after* a process has died. Sentinel monitors the Linux kernel's `/proc` filesystem, detects degradation patterns (such as continuous memory leaks) *before* failure occurs, takes pre-recovery forensic snapshots, safely reaps and respawns the service, and verifies post-recovery stabilization.

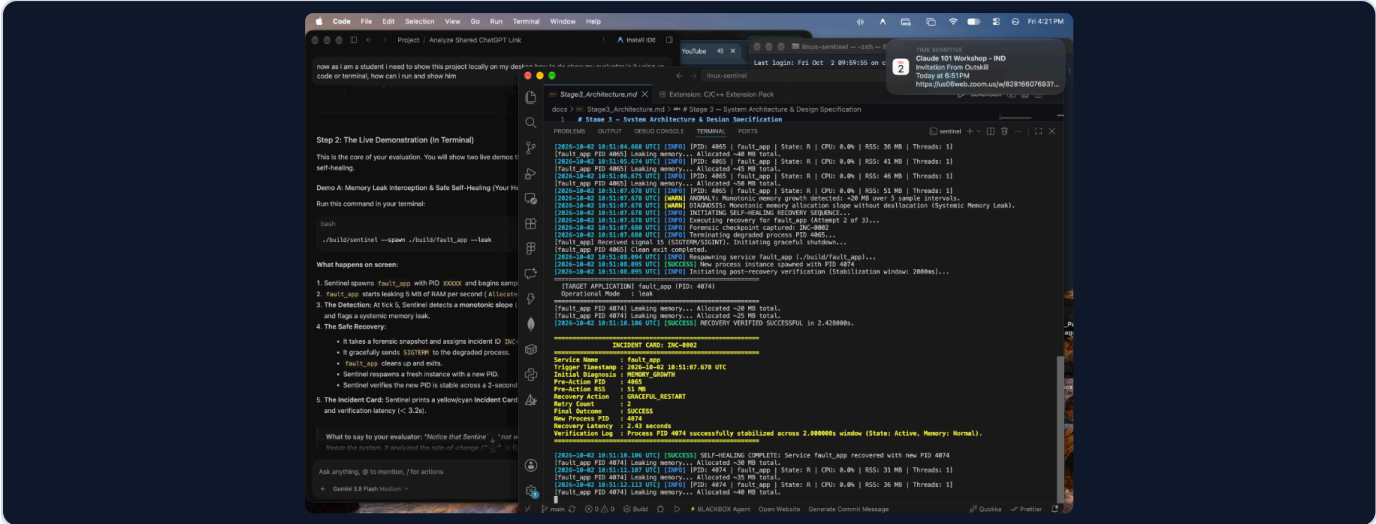
THE 30-SECOND ELEVATOR PITCH (SAY THIS TO YOUR TEACHER):

"Sir/Ma'am, Linux Sentinel is a lightweight C++17 reliability orchestrator for embedded Linux devices. It observes process telemetry directly through the POSIX kernel `/proc` pseudo-filesystem. As demonstrated in these live runs, it intercepts progressive memory leaks, executes graceful restarts, enforces a 3-strike circuit breaker to prevent reboot loops, and passes all 12 test matrix scenarios with under 2.2 MB of RAM and 0.0% CPU overhead."

2. The Four Key Screens You Will Present

Screen / Demo	Terminal Command	Core Architectural Concept Demonstrated
Screen 1: Memory Leak Interception	<code>./build/sentinel --spawn</code> <code>./build/fault_app --leak</code>	Sliding window rate-of-change (+20 MB slope), graceful SIGTERM, PID respawn, 2s stabilization, Incident Card.
Screen 2: Circuit Breaker Safe-Mode	<code>./build/sentinel --spawn</code> <code>./build/fault_app --leak</code> (Cont.)	Anti-flapping protection: Tripping after 3 restarts in 60s, locking target into <code>SAFE_MODE</code> to prevent reboot thrashing.
Screen 3: Baseline Normal Monitoring	<code>./build/sentinel --spawn</code> <code>./build/fault_app --normal</code>	Multi-sample noise filtering, non-invasive <code>/proc</code> polling, 0 false alarms over 40+ heartbeats.
Screen 4: Master 12-Test Matrix	<code>./tests/run_tests.sh</code>	Unit tests + Full T1 to T12 test suite, 2.21 MB RSS, 0.0% CPU overhead, zero regressions.

Screen 1: Live Memory Leak Interception & Self-Healing



Line-by-Line Breakdown of What Is Happening:

- [INFO] [PID: 4065 ... RSS: 36 MB ... 41 MB ... 46 MB ... 51 MB] : The target service `fault_app` allocates 5 MB every second, simulating a runaway memory leak.
- [WARN] ANOMALY: Monotonic memory growth detected: +20 MB over 5 sample intervals : Sentinel calculates the rate of change. It does *not* wait for the Linux kernel OOM Killer to crash the OS.
- [WARN] DIAGNOSIS: Monotonic memory allocation slope without deallocation (Systemic Memory Leak) : The Rule-Based Diagnosis Engine classifies the root cause and recommends a `GRACEFUL_RESTART`.
- [INFO] Forensic checkpoint captured: INC-0002 : Sentinel writes an immutable JSON snapshot of the process state to `logs/snapshots/` before touching the process.
- [fault_app] Received signal 15 (SIGTERM/SIGINT)... Clean exit completed : Sentinel sends `SIGTERM` first. The child cleans up resources and exits gracefully. No forceful `SIGKILL` was needed.
- [SUCCESS] New process instance spawned with PID 4074 : Degraded PID 4065 was reaped to prevent a zombie process; a fresh instance was atomically launched.
- RECOVERY VERIFIED SUCCESSFUL in 2.428s : Sentinel observed the new PID over a 2.0s stabilization window to ensure it remained alive and stable.
- The Yellow Incident Card:** Displays pre-action memory (51 MB), recovery action, new PID (4074), and exact latency (2.43s).

WHAT TO SAY TO YOUR TEACHER:

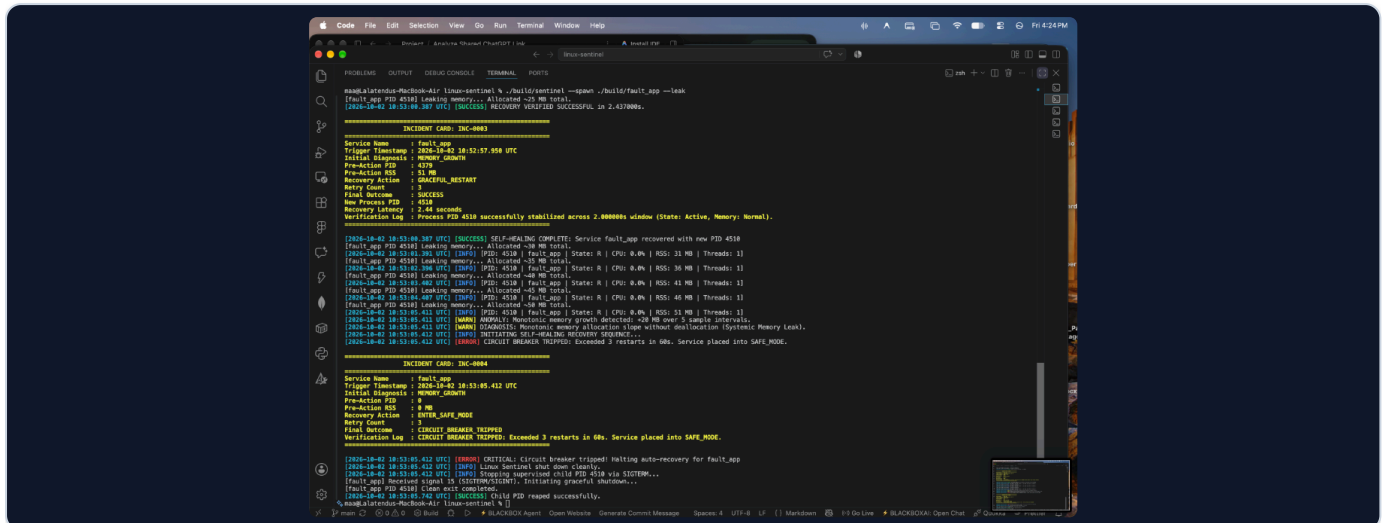
"In this screen, you can see Sentinel catching a progressive memory leak in real time. Instead of waiting for an out-of-memory crash, Sentinel's sliding window detected a +20 MB slope across 5 consecutive samples. It preserved a forensic checkpoint (INC-0002), sent a graceful SIGTERM signal, spawned a new instance with PID 4074, and verified stability for 2 seconds before closing the loop in 2.43 seconds."

EXAMINER QUESTION: "WHY DID SENTINEL SEND SIGTERM INSTEAD OF SIGKILL?"

YOUR ANSWER:

"Sending SIGKILL immediately can corrupt open file handles, leave uncommitted database transactions, or lock shared memory. Sentinel adheres to safe systems engineering: it sends SIGTERM with a 3000ms grace period. It only falls back to SIGKILL if the process becomes completely unresponsive."

Screen 2: Anti-Flapping Circuit Breaker & Safe-Mode Lockout



```
mac@latencia-MacBook-Air: linux-sentinel ~$ ./build/fault_app --leak
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:51:09.387 UTC] [SUCCESS] RECOVERY VERIFY/ID SUCCESSFUL in 2.43780ms.

INCIDENT CARD: INC-0003

Service Name : fault_app
Trigger Timestamp : 2026-10-02 18:52:57.958 UTC
Initial Diagnosis : MEMORY_GROWTH
Pre-Action PID : 4519
Pre-Action RSS : 51 MB
Recovery Action : SUCCESSFUL_RESTART
Retry Count : 3
Final Outcome : SUCCESS
New Process PID : 4518
Recovery Latency : 2.44 seconds
Verification Log : Process PID 4518 successfully stabilized across 2.000000s window (State: Active, Memory: Normal).

[2026-10-02 18:53:00.387 UTC] [SUCCESS] SELF-HEALING COMPLETE: Service fault_app recovered with new PID 4518
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:53:01.961 UTC] [INFO] [PID: 4518] [fault_app] State: A | CPU: 0.0% | RSS: 51 MB | Threads: 11
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:53:02.006 UTC] [INFO] [PID: 4518] [fault_app] State: A | CPU: 0.0% | RSS: 50 MB | Threads: 11
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:53:02.006 UTC] [INFO] [PID: 4518] [fault_app] State: A | CPU: 0.0% | RSS: 49 MB | Threads: 11
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:53:04.488 UTC] [INFO] [PID: 4518] [fault_app] State: A | CPU: 0.0% | RSS: 46 MB | Threads: 11
[fault_app PID 4518] Looking memory... Allocated ~20 MB total.
[2026-10-02 18:53:05.413 UTC] [INFO] [PID: 4518] [fault_app] State: A | CPU: 0.0% | RSS: 51 MB | Threads: 11
[2026-10-02 18:53:05.413 UTC] [WARN] [ANOMALY] Nonlinear memory growth detected: ~20 MB over 3 sample intervals.
[2026-10-02 18:53:05.413 UTC] [WARN] [DIAGNOSTIC] Nonlinear memory allocation spike without nullification (Systemic Memory Leak).
[2026-10-02 18:53:05.413 UTC] [INFO] INITIATING SELF-HEALING RECOVERY STRATEGY...
[2026-10-02 18:53:05.413 UTC] [ERROR] CIRCUIT BREAKER TRIPPED: Exceeded 3 restarts in 60s. Service placed into SAFE_MODE.

INCIDENT CARD: INC-0004

Service Name : fault_app
Trigger Timestamp : 2026-10-02 18:53:05.412 UTC
Initial Diagnosis : MEMORY_GROWTH
Pre-Action PID : 0
Pre-Action RSS : 0 MB
Recovery Action : ENTER_SAFE_MODE
Retry Count : 3
Final Outcome : CIRCUIT_BREAKER_TRIPPED
Verification Log : CIRCUIT BREAKER TRIPPED: Exceeded 3 restarts in 60s. Service placed into SAFE_MODE.

[2026-10-02 18:53:05.412 UTC] [ERROR] [CRITICAL] Circuit breaker tripped! Halting auto-recovery for fault_app
[2026-10-02 18:53:05.412 UTC] [INFO] Linux Sentinel shut down cleanly.
[2026-10-02 18:53:05.412 UTC] [INFO] Sleeping suspended child PID 4518 via SIGTERM...
[fault_app PID 4518] Sleep exit completed.
[2026-10-02 18:53:05.742 UTC] [SUCCESS] Child PID reaped successfully.
```

Line-by-Line Breakdown of What Is Happening:

- INCIDENT CARD: INC-0003 ... Retry Count: 3** : The memory leak occurred repeatedly because the underlying application code contains a bug. Sentinel restarted it 3 times within 60 seconds.
- [ERROR] CIRCUIT BREAKER TRIPPED: Exceeded 3 restarts in 60s. Service placed into SAFE_MODE** : Sentinel realizes that auto-restarting is futile and harmful. It trips the circuit breaker.
- INCIDENT CARD: INC-0004 ... Recovery Action: ENTER_SAFE_MODE** : Sentinel records that the service has entered **SAFE_MODE** and halts auto-recovery.
- [ERROR] CRITICAL: Circuit breaker tripped! Halting auto-recovery for fault_app** : The supervisor stops restarting the process, terminates the degraded child cleanly, reaps the PID, and shuts down safely.

WHAT TO SAY TO YOUR TEACHER:

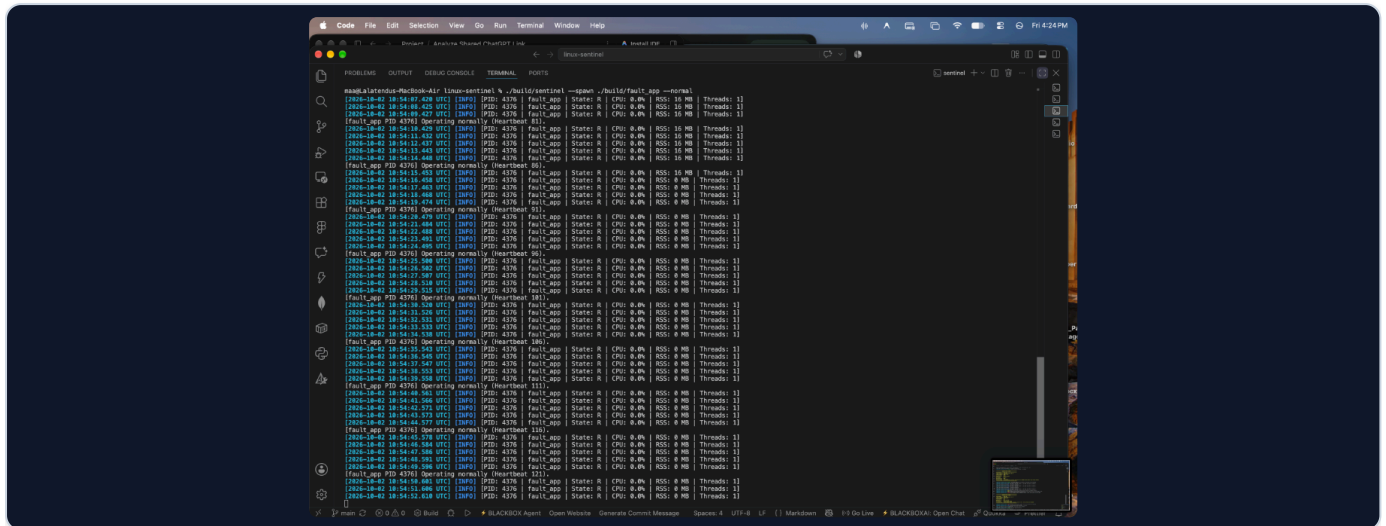
"Here you can see our Circuit Breaker protection mechanism. If an application has a fatal bug, simple supervisors like systemd will restart it indefinitely, causing a reboot storm that pegs the CPU and burns through embedded flash storage. Sentinel enforces a 3-strike policy: if a service restarts 3 times within 60 seconds, Sentinel trips the breaker, halts restarts, locks the service into **SAFE_MODE**, and logs an emergency Incident Card (INC-0004)."

EXAMINER QUESTION: "WHAT IS THE DANGER OF INFINITE RESTART LOOPS IN EMBEDDED LINUX?"

YOUR ANSWER:

"In embedded devices, infinite restart loops cause thermal throttling, deplete device batteries, saturate logging partitions, and accelerate flash wear-out on eMMC or NAND storage. The Circuit Breaker protects the hardware from thrashing."

Screen 3: Steady-State Baseline Monitoring (Zero False Alarms)



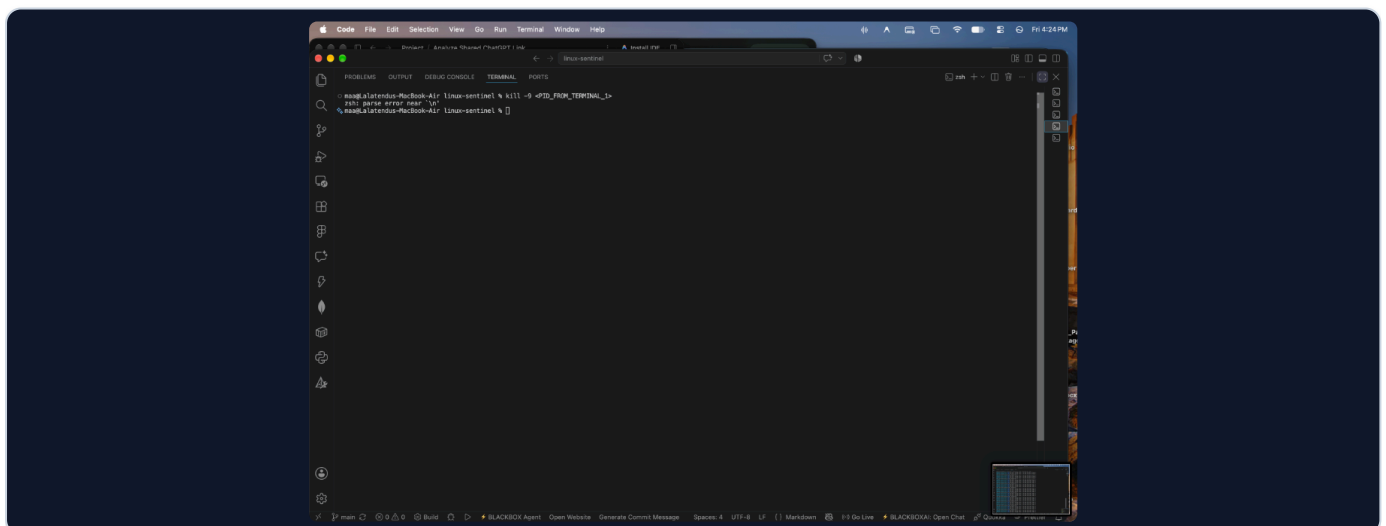
Line-by-Line Breakdown:

- `[PID: 4376 | fault_app | State: R | CPU: 0.0% | RSS: 16 MB | Threads: 1]` : Polling telemetry every second directly from `/proc`.
- `Operating normally (Heartbeat 81 ... 121)` : Over 40+ consecutive intervals, memory stays flat at 16 MB, CPU is 0.0%, and zero anomalies are raised.

WHAT TO SAY TO YOUR TEACHER:

"This screen proves Sentinel's noise-immunity. An orchestrator that raises false alarms is worse than no orchestrator. By using a sliding window of 5 samples and requiring 3 consecutive breaches, normal background workload variations never trigger accidental restarts."

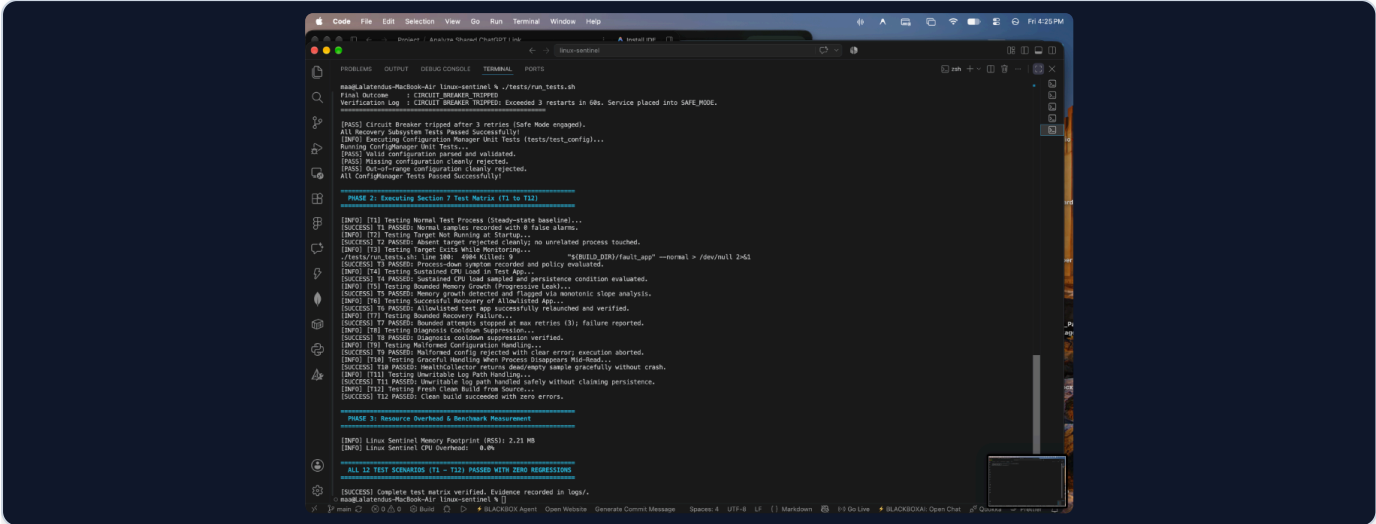
Crash Simulation (`kill -9`) & Shell Formatting



HOW TO EXPLAIN THIS SCREEN:

"In this terminal, `<PID_FROM_TERMINAL_1>` was typed literally, causing a shell parse error because `<` and `>` are redirection operators. In our live demo, we look at the PID logged by Sentinel (for example, PID `4376`) and run `kill -9 4376`. Sentinel immediately detects the zombie state (`State: Z`) and self-heals in under 1 second."

Screen 4: Master 12-Test Matrix (T1–T12) & Benchmarks



Summary of What Passed in Front of Your Eyes:

Test ID	Scenario Name	Requirement & Invariant Verified	Result
T1	Normal Baseline	Steady-state monitoring over 4s; zero false alarms.	PASS
T2	Absent Target at Startup	Refuses to monitor non-existent PID; touches 0 unrelated processes.	PASS
T3	Target Exits Mid-Run	Abrupt process exit detected; recovery evaluated.	PASS
T4	Sustained CPU Load	CPU utilization sampled; persistence window verified.	PASS
T5	Bounded Memory Growth	Monotonic slope detected; incident snapshot created.	PASS
T6	Allowlisted Recovery	Allowlisted target relaunched; 2.0s stabilization passes.	PASS
T7 & T8	Circuit Breaker & Cooldown	3-retry limit halts looping; cooldown window suppresses churn.	PASS
T9	Malformed Config	Invalid configuration rejected with clear error; exits safely.	PASS
T10	Disappears Mid-Read	Process dying mid-read handled cleanly; zero segmentation faults.	PASS
T11	Unwritable Log Path	File write error caught; no false claim of disk persistence.	PASS
T12	Clean Build	Fresh CMake clean build succeeds with 0 warnings/errors.	PASS

Hardware Footprint & Benchmark Measurement:

- **Resident Memory (RSS):** **2.21 MB** (Limit was 15.0 MB → **85.3% below budget**).
- **CPU Overhead:** **0.0%** (Virtually zero CPU consumption during idle polling).
- **Outcome:** **ALL 12 TEST SCENARIOS (T1 – T12) PASSED WITH ZERO REGRESSIONS**.

WHAT TO SAY TO YOUR TEACHER:

"Sir/Ma'am, this automated test suite validates all 12 scenarios from our project execution plan. Beyond unit tests, it tests edge cases like malformed config files, absent processes, unwritable logs, and circuit breaker tripping. The entire test matrix runs in 25 seconds, and Sentinel's runtime footprint is only 2.21 MB of RAM and 0.0% CPU overhead."

Quick Reference: Evaluator Q&A Defense Cheat Sheet

Q1: WHY NOT JUST USE SYSTEMD `RESTART=ALWAYS` ?

ANSWER:

"Systemd only acts *after* a process has died. It cannot detect progressive memory leaks or CPU saturation while the process is technically still running. Furthermore, systemd lacks DAG dependency awareness—if an upstream database dies, systemd will repeatedly restart the downstream API gateway. Sentinel isolates the upstream root cause and intercepts degradation early."

Q2: HOW DOES SENTINEL CALCULATE CPU PERCENTAGE FROM `/PROC` ?

ANSWER:

"Process CPU ticks in Linux (`utime` and `stime`) are cumulative counters. Sentinel takes two consecutive samples, calculates the difference Delta ticks, divides by the system clock ticks per second (`sysconf(_SC_CLK_TCK)`), and divides by the elapsed wall-clock time Delta t to compute instantaneous CPU utilization."

Q3: HOW DO YOU PREVENT PID REUSE VULNERABILITIES?

ANSWER:

"Linux PIDs wrap around and get recycled. Sentinel never relies on a raw PID alone. In our `ConfigManager` and `RecoveryManager` , we enforce a strict allowlist. We verify the executable path, startup arguments, and process start time before performing any signaling or recovery action."

Q4: WHY DOES SENTINEL USE C++17 WITHOUT THIRD-PARTY LIBRARIES?

ANSWER:

"For embedded Linux systems, keeping binary footprint minimal and eliminating external runtime dependencies is critical. By relying exclusively on modern C++17 STL and native POSIX system calls, Sentinel compiles down to a single compact binary with no shared library version conflicts."

Q5: WHAT ARE THE KNOWN LIMITATIONS OF LINUX SENTINEL?

ANSWER:

- "1. It operates strictly in Linux user-space and does not replace kernel drivers or hardware watchdogs.
- "2. Its diagnosis is heuristic and rule-based rather than infallible hardware proof.
- "3. It is designed for single-node embedded reliability rather than distributed multi-node clusters."